CS2323 Computer Architecture

Homework-1: RISC-V Assembly Encoding
MA24BTECH11022

Q1. Write equivalent machine code (in hexadecimal) for the given assembly instructions, by highlighting the various fields in the 32-bits of the instruction: [10 marks]

(a) `addi x15, x22, -45`
The given instruction is an I-type RISC-V instruction, which consists of a 12-bit immediate, the `rs1` register, `funct3`, the destination register `rd`, and the `opcode`. The immediate value is -45, which is represented in 12-bit two's complement form as `111111010011`. The source register is `x22`, so `rs1` = `10110`. `funct3` for `addi` is `000`. The destination register is `x15`, so `rd` = `01111`. The opcode for `addi` is `0010011`.

immediate[11:0]	rs1	funct3	rd	opcode
111111010011	10110	000	01111	0010011

Converting to hexadecimal by grouping the binary machine code in groups of 4 we get `0xFD3B0793`.

(b) `and x23, x8, x9`
The given instruction is an R-type RISC-V instruction, which consists of `funct7`, the `rs2` register, the `rs1` register, `funct3`, the destination register `rd`, and the `opcode`. The `funct7` for `and` is `0000000`. The source register `rs1` is `x8`, so `rs1` = `01000`. The source register `rs2` is `x9`, so `rs2` = `01001`. `funct3` for `and` is `111`. The destination register is `x23`, so `rd` = `10111`. The opcode for `and` is `0110011`.

funct7	rs2	rs1	funct3	rd	opcode
0000000	01001	01000	111	10111	0110011

Converting to hexadecimal by grouping the binary machine code in groups of 4 we get `0x00947BB3`.

(c) `blt x2, x11, 240`
The given instruction is a B-type RISC-V instruction, which consists of a 12-bit immediate, the `rs1` register, `rs2` register, `funct3`, and the `opcode`. The immediate value is 240, which is represented in 13-bit binary form as `0000011110000`. The source register is `x2`, so `rs1` = `00010`. The source register is `x11`, so `rs2` = `01011`. `funct3` for `blt` is `100`. The opcode for `blt` is `1100011`.

imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode
0	000111	01011	00010	100	1000	0	1100011

Converting to hexadecimal by grouping the binary machine code in groups of 4 we get `0x0EB14863`.

(d) `sd x19, -54(x1)`
The given instruction is an S-type RISC-V instruction, which consists of a 13-bit immediate, the `rs1` register, the `rs2` register, `funct3`, and the `opcode`. The immediate value is -54, which is represented in 12-bit two's complement form as `111111001010`. The base register is `x1`, so `rs1` = `00001`. The source register is `x19`, so `rs2` = `10011`. `funct3` for `sd` is `011`. The opcode for `sd` is `0100011`.

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
1111110	10011	00001	011	01010	0100011

Converting to hexadecimal by grouping the binary machine code in groups of 4 we get `0xFD30B523`.

(e) `jal x3, -10116`
The given instruction is a J-type RISC-V instruction, which consists of a 21-bit immediate, the destination register `rd`, and the `opcode`. The immediate value is -10116, which is represented in 21-bit two's complement form as `1111110110001101100`. The destination register is `x3`, so `rd` = `00011`. The opcode for `jal` is `1101111`.

imm[20]	imm[10:1]	imm[11]	imm[19:12]	rd	opcode
1	0000111110	1	11111101	00011	1101111

Converting to hexadecimal by grouping the binary machine code in groups of 4 we get `0x87DFD1EF`.

Q2. For various pseudo instructions shown below, write their equivalent using a maximum of 2 real instructions. [4 marks]
Note: The instruction li represents the pseudo instruction load immediate.

(a) `li x5, 0xFFFFFFFFFFFFFFFF`
`0xFFFFFFFFFFFFFFFF` is hexadecimal for -1, which lies in the range for `addi`.
`addi x5, x0, -1`

(b) `li x5, 132`
132 lies in 12-bit signed range which is [-2048, 2047].
`addi x5, x0, 132`

(c) `li x5, 2134`
2134 is greater than 2047, so we cannot use `addi` instruction directly as we did in the previous questions.
2134 = 4096 - 1962, so we can load 4096 using `lui` instruction and then subtract 1962 from it using `addi`.
`lui x5, 1`
`addi x5, x5, -1962`

(d) `li x5, 0x000000002345abcd`
`0x000000002345abcd` is 591,768,525 which clearly does not lie in the range of `addi`.
Observe that $591,768,525 = (144475 * 4096) - 1075$. So we can load $144475 * 4096$ using `lui` and then use `addi` to add 973 into it.
`lui x5, 144475`
`addi x5, x5, -1075`

Q3. Convert the given instructions in hex to their corresponding assembly code [6 marks]

(a) `0x0019F233`

`0x0019F233` = `0000000 00001 10011 111 00100 0110011`

The opcode is `0110011`, so this is an R-type instruction.

funct7	rs2	rs1	funct3	rd	opcode
0000000	00011	10011	111	00100	0110011

$$\text{rs2} = \texttt{00011} = x1,$$
$$\text{rs1} = \texttt{10011} = x19,$$
$$\text{rd} = \texttt{00100} = x4.$$

Since `funct7`=0000000, `funct3`=111, the instruction is `and`. Therefore, the instruction is `and x4, x19, x1`.

(b) 0x06B4D763

`0x06B4D763` = `0000011 01011 01001 101 1110 0 1100011`

The opcode is `1100011`, so this is a B-type instruction.

imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode
0	000011	01011	01001	101	1110	0	1100011

$$\text{rs2} = \texttt{01011} = x11,$$
$$\text{rs1} = \texttt{01001} = x9,$$

For a B-type instruction, funct3=`101` corresponds to `BGE`.
The immediate is `0|000011|0|1110|0` = `0000011011100` = 110. So, the instruction is `bge x9, x11, 110`.

(c) 0x0169CF93

`0x0169CF93` = `000000010110 10011 100 11111 0010011`

The opcode is `0010011`, so this is an I-type instruction.

imm[11:0]	rs1	funct3	rd	opcode
000000010110	10011	100	11111	0010011

$$\text{rs1} = \texttt{10011} = x19,$$
$$\text{rd} = \texttt{11111} = x31$$

Since `funct3`=`100`, the instruction is `XORI`.
The immediate is `000000010110` = 28. So, the instruction is `xori x31, x19, 22`.